Geavanceerede Computerarchitectuur Project

Jonas Van Nieuwenhuyze, Gil Nimmegeers

1 Introduction

In this report, we present a comparative analysis of game performance between regular C++ and CUDA implementations for raycasting applications. We made a project on a raycaster in python during our second Bachelor's year. For this project, we aimed to evaluate the performance improvement achievable by implementing a raycaster in C++ with CUDA.

2 Implementation

The C++ implementation utilizes SDL for window management and rendering and SDL_image for image loading. The code can be found at: `https://github.com/Jonas-VN/CUDA-Raycaster` (Some code snippets are provided here for a more thorough explanation)

2.1 Raycasting

Raycasting is a rendering technique used to create a 3D perspective in 2D games by casting rays from the player's viewpoint to simulate depth and spatial awareness. For every column on the screen the direction of a ray is calculated. This direction is used to calculate the distance to the first wall intersection. The height of the wall is correlated to the distance to the wall. This is done in a for loop for every column, but since we are using CUDA it is possible to do these calculations in parallel.

2.2 Texture Memory Optimization

Texture memory in CUDA offers optimized memory access patterns tailored for texture fetching operations. In this implementation, we loaded texture data into texture memory to facilitate efficient texture mapping during raycasting. Texture memory provides hardware-assisted interpolation and caching mechanisms, enabling fast and efficient texture sampling. By leveraging texture memory, we minimized memory access latency and improved rendering performance, especially in scenarios with high texture sampling requirements. Additionally, texture memory's caching properties enhance memory bandwidth utilization, further enhancing overall performance.
Since CUDA 12.0 texture references are not possible anymore, now CUDA only supports texture objects which gives more control to the developer. With texture objects, the developer itself needs to set metadata for the texture (cudaResourceDesc object), such as: the total size of the texture in bytes, bits per color channel, etc. Therefore we implemented texture objects instead of texture references.

To load textures, we utilized the SDL_Image library to load image files into CPU memory. We then converted the image format to RGBA32, suitable for rendering in SDL2 (inside the loadImage function). Subsequently, we transferred the texture data from CPU memory to GPU memory using CUDA's memory copy functions (cudaMemcpy). Once the texture data is in GPU memory, we created a CUDA texture object (cudaTextureObject) and bound it to the texture data, enabling efficient texture sampling during raycasting.

```
    Uint32* texture = loadImage("src/wall.png");
    Uint32* textureData;
    cudaMalloc((void**)&textureData, TEXTURE_HEIGHT * TEXTURE_WIDTH
     * sizeof(Uint32));
    cudaMemcpy(textureData, texture, TEXTURE_HEIGHT * TEXTURE_WIDTH
     * sizeof(Uint32), cudaMemcpyHostToDevice);

    cudaResourceDesc resDesc;
    memset(&resDesc, 0, sizeof(resDesc));
    resDesc.resType = cudaResourceTypeLinear;
    resDesc.res.linear.devPtr = textureData;
    resDesc.res.linear.desc.x = 32; // bits per channel
    resDesc.res.linear.sizeInBytes = TEXTURE_HEIGHT * TEXTURE_WIDTH
     * sizeof(Uint32);

    cudaTextureDesc texDesc;
    memset(&texDesc, 0, sizeof(texDesc));
    texDesc.readMode = cudaReadModeElementType;

    cudaTextureObject_t gpuTexture = 0;
    cudaCreateTextureObject(&gpuTexture, &resDesc, &texDesc, NULL);
```

Listing 1: Texture object in CUDA

2.3 Class methods on CPU and GPU

Some of the class methods are used on the CPU and GPU, for example the Direction and Coordinate class methods. We need the direction and coordinate of the player which is used for player movement on the CPU. We also need these classes on the GPU for the direction of a ray and the coordinate where a ray intersects with a wall.
Enabling the use of methods both on the CPU and GPU is very easy. They are accessible by the CPU by default, but when adding **__device__** to make them also accessible on the GPU, it is no longer accessible on the CPU. Therefore **__device__** and **__host__** should both be added to the method declaration to make this method available on both the CPU and GPU.

```
    struct Direction {
    public:
        ...
        __host__ __device__ void normalize() {
            double magnitude = sqrt(x * x + y * y);
            if (magnitude != 0.0) {
```

```
                x /= magnitude;
                y /= magnitude;
            }
        }
        ...
    }
```

Listing 2: Example method accessible on CPU and GPU

This method for example can normalize a direction both on the CPU and GPU.

2.4 Pointer management on GPU

```
class Player {
public:
    Coordinate* coordinate;
    Direction* direction;
    Camera* camera;
    ...
}

class Camera {
public:
    Direction* direction;
    const int pov;
    const double distanceToPlayer;
    ...
}
```

Listing 3: Example pointer management

A player object is used on the GPU. However since the player attributes change every frame, they should be copied to the GPU every frame to ensure it reflects the latest data. This may look like a lot of data to copy every frame, but when taking a closer look, it turns out it's not. Given that the attributes are dynamically allocated, the player object remains constant on the GPU, necessitating the need to copy it only once. There are no statically allocated attributes and the pointers on the GPU stay the same. Similarly, the camera's statically allocated attributes and single pointer also remain constant. Only the data where the pointers point to of the dynamically allocated attributes require updating.

```
// Make objects for on the GPU
Player* gpuPlayer;
Camera* gpuCamera;
Direction* gpuPlayerDirection;
Coordinate* gpuPlayerCoordinate;
Direction* gpuCameraDirection;

// Allocate memory for these objects
cudaMalloc((void**)&gpuPlayer, sizeof(Player));
cudaMalloc((void**)&gpuCamera, sizeof(Camera));
```

```
11      cudaMalloc((void**)&gpuPlayerDirection, sizeof(Direction));
12      cudaMalloc((void**)&gpuPlayerCoordinate, sizeof(Coordinate));
13      cudaMalloc((void**)&gpuCameraDirection, sizeof(Direction));
14
15      // The player & camera are technically constant since only the
        pointers are relevant or the other fields are constant
16      cudaMemcpy(gpuPlayer, player, sizeof(Player),
        cudaMemcpyHostToDevice);
17      cudaMemcpy(gpuCamera, player->camera, sizeof(Camera),
        cudaMemcpyHostToDevice);
18
19      // Populate pointers (these can be copied in advance because
        the gpuPlayer & gpuCamera never changes and thus the pointers
        stay relevant)
20      cudaMemcpy(&(gpuPlayer->direction), &gpuPlayerDirection, sizeof
        (Direction*), cudaMemcpyHostToDevice);
21      cudaMemcpy(&(gpuPlayer->coordinate), &gpuPlayerCoordinate,
        sizeof(Coordinate*), cudaMemcpyHostToDevice);
22      cudaMemcpy(&(gpuPlayer->camera), &gpuCamera, sizeof(Camera*),
        cudaMemcpyHostToDevice);
23      cudaMemcpy(&(gpuCamera->direction), &gpuCameraDirection, sizeof
        (Direction*), cudaMemcpyHostToDevice);
```

Listing 4: Allocating and copying the classes to the GPU

The above code (see Listing 4) only needs to be run once during startup. During the game loop, updates are only required for the player's direction, coordinates, and camera direction. This is because all pointers retain the same address.

```
1       while (gameIsRunning) {
2           ...
3           cudaMemcpy(gpuPlayerDirection, player->direction, sizeof(
        Direction), cudaMemcpyHostToDevice);
4           cudaMemcpy(gpuPlayerCoordinate, player->coordinate, sizeof(
        Coordinate), cudaMemcpyHostToDevice);
5           cudaMemcpy(gpuCameraDirection, player->camera->direction,
        sizeof(Direction), cudaMemcpyHostToDevice);
6           ...
7       }
```

Listing 5: Copying playerdata to the GPU every frame

2.5 Constant memory

The only data that is stored in constant memory, is the map data since it never changes. Nothing else is stored in constant memory since it varies a lot due to player movement.

2.6 Processing User Input

In our implementation, we utilized SDL for handling user input events, such as keyboard and mouse interactions. SDL provides a platform-independent

interface for handling input events, making it suitable for our cross-platform requirements. We implemented a handlekeys() function to process keyboard input, detecting key presses and updating the player's movement accordingly. Additionally, we utilized SDL's event handling mechanism to capture mouse motion events, allowing the player to rotate the view direction in response to mouse movements.

2.7 Rendering

Since SDL functions are not available on the GPU, we decided to control every pixel individually stored in an array. First we populate this array on the gpu. Afterwards we copy it back to the GPU when the kernel has finished. Lastly we update the surface to show these pixels on the screen.

2.8 Game loop

In the game loop we fist calculate the FPS and the delta time (time to render 1 frame). Next, we copy the player data to the GPU asynchronously, do the raycasting and copy the pixel data back to the CPU asynchronously. While the GPU is busy we keep checking for key inputs and update the player accordingly. These changes to the player only become visible on the next frame when the player data is copied again. After the GPU has finished, we update the screen and jump back to the top of the loop.

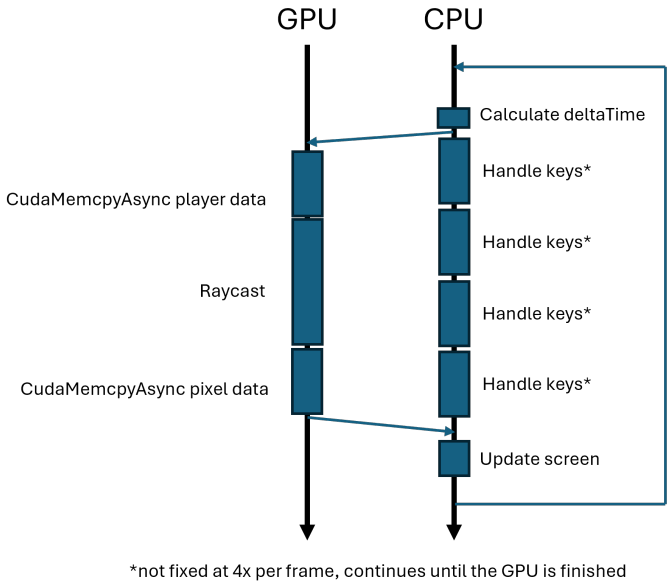


Figure 1: Game loop layout

3 Results

The code is written in such a way that it is easy to test different setups. We can choose between using the CPU/GPU and whether or not to use textures. When using textures on the GPU there is also the option to use a texture object. Figure 2 and figure 3 show that the GPU beats the CPU in every setup. The GPU also has more consistent FPS between standing still and walking around. Using textures on the GPU does not impact the performance as much as on the CPU. Using texture objects instead of a regular array on the GPU bumps up the FPS with about 10 in both scenarios. As expected the performance of the python code from the second years project is a lot worse.
Choosing an optimal block size is crucial when invoking a kernel. Therefore, we tested the performance for different block sizes, ranging from 2 threads to 1024 threads. Surprisingly, the best performance was achieved with a block size of 32 or 64 threads (see Figure 4). This result may be attributed to the fact that having a lower number of threads per block hinders the potential for thread collaboration. While adding more threads per block could theoretically expedite execution, this was not the case in our scenario. The reason lies in the nature of our raycasting algorithm.
When using 1024 threads per block on a screen with a width of 1200 pixels, there is a higher likelihood of encountering sudden disparities in distances to walls. For instance, one half of the screen may have a distance of 1m to a wall, while the other half may be 8m away. This abrupt change creates two distinct groups of threads within a block, each writing to memory locations significantly distant from each other. As a result, memory coalescing, which optimizes memory accesses by grouping contiguous threads, is hindered. Consequently, even with more threads per block, the benefits of parallelism are offset by the lack of memory coalescing, leading to suboptimal performance.
Having 32 or 64 threads per block is the golden middle ground, as this reduces the likelihood of such jumps and still provides enough threads to collaborate effectively.

4 Conclusion

Using the GPU for raycasting applications has a real benefit for the performance. The FPS is higher and more stable. Using textures also does not impact the performance that much on the GPU.

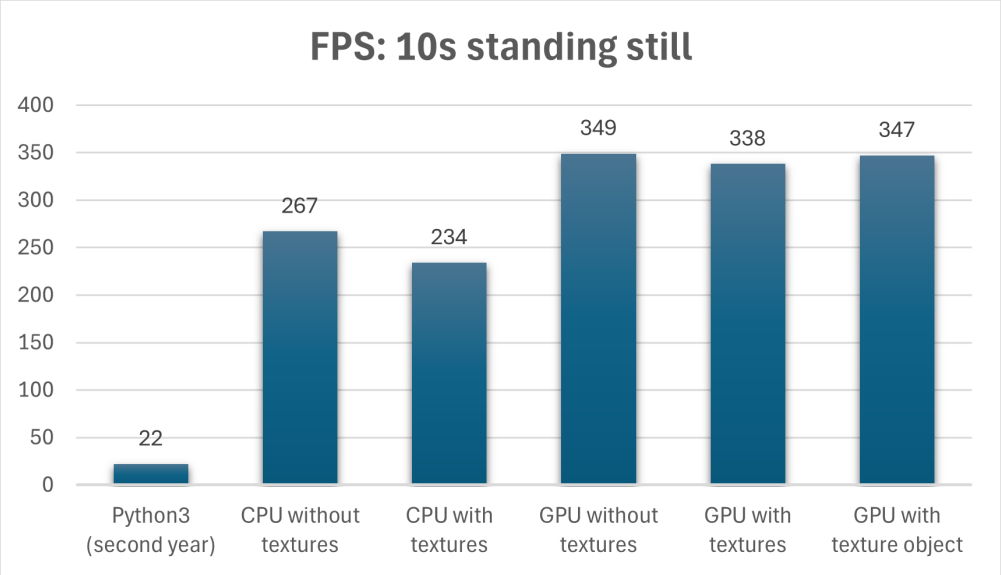


Figure 2: FPS while standing still

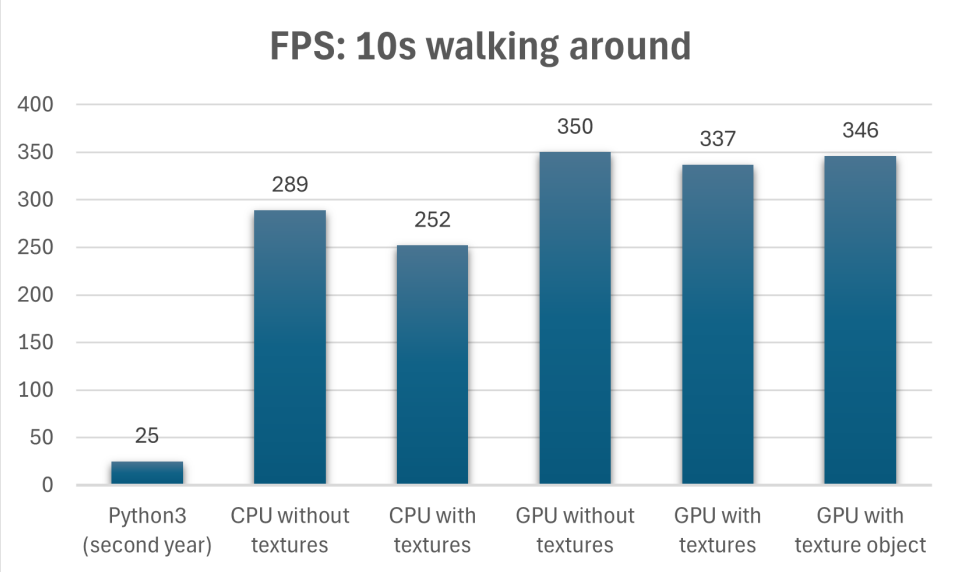


Figure 3: FPS while walking around

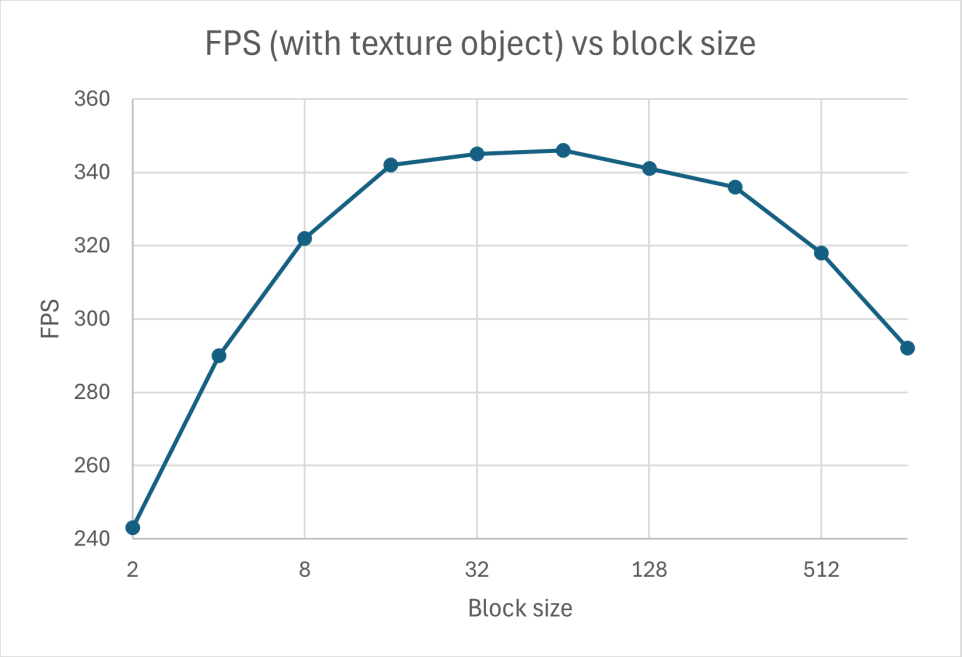


Figure 4: FSP vs block size